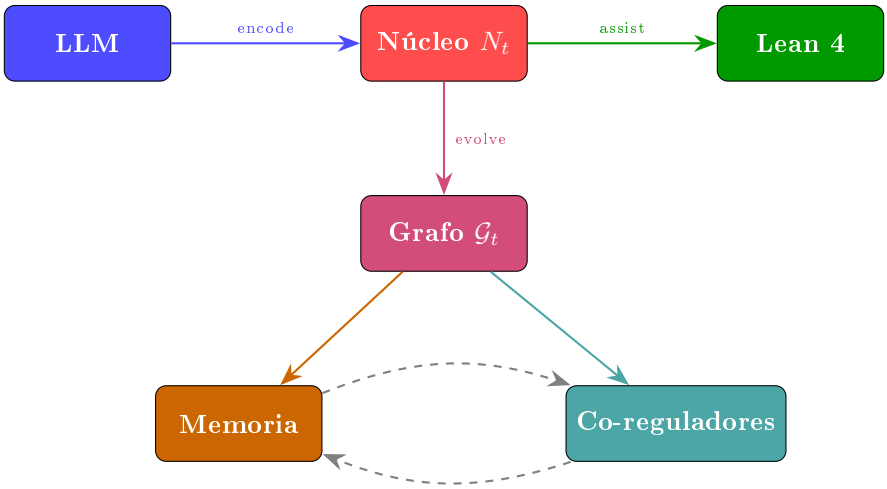
Núcleo Lógico Evolutivo

como Sistema Evolutivo con Memoria

Integración del marco teórico de
Ehresmann & Vanbremeersch



Autores

Leonardo Jiménez Martínez
Universidad Nacional Autónoma de México

Claude (Anthropic)
Modelo de Lenguaje con Capacidades de Razonamiento Formal

Febrero 2026 · Versión 7.0

Documento Unificado: Sistemas Evolutivos con Memoria,
Co-reguladores, Complejificación y Rigor Científico

Contents

1 Introducción: De Grafo Dinámico a Sistema Evolutivo con Memoria **2**
1.1 Motivación . . . 2
1.2 Estructura del Documento . . . 2

2 Preliminares: El Marco MES de Ehresmann & Vanbremeersch **2**
2.1 Patrones y Colímites . . . 3
2.2 Categorías Jerárquicas . . . 3
2.3 El Principio de Multiplicidad . . . 3
2.4 Sistemas Evolutivos . . . 4
2.5 Complejificación respecto a una Opción . . . 4
2.6 Sistemas Evolutivos con Memoria y Co-reguladores . . . 5

3 El Grafo de Skills como Sistema Evolutivo Jerárquico **5**
3.1 Reformulación del Grafo Categórico . . . 5
3.2 El Sistema Evolutivo de Skills . . . 6
3.3 Multiplicidad en el Grafo de Skills . . . 7

4 Red de Co-reguladores del NLE **7**
4.1 Del Agente RL Monolítico a una Red de Co-reguladores . . . 7
4.2 Paisajes de los Co-reguladores . . . 8
4.3 Ciclo Operativo de un Co-regulador . . . 8

5 Arquitectura de Memoria del NLE **9**
5.1 Estructura de la Memoria . . . 9
5.2 Tipos de Memoria en el NLE . . . 10

6 Evolución del Grafo como Complejificación **10**
6.1 Las Acciones del Núcleo como Opciones . . . 10
6.2 La Operación `merge` como Ligadura de Patrón . . . 10
6.3 Emergencia de Enlaces Complejos . . . 11

7 Plasticidad y Robustez vía el Principio de Multiplicidad **11**
7.1 Robustez por Descomposiciones Múltiples . . . 11
7.2 Plasticidad por Complejificación . . . 11
7.3 Fracturas y Reparación . . . 12

8 Propiedades Formales **12**
8.1 Axiomas Extendidos . . . 12
8.2 Teoremas . . . 12

9 Integración con la Arquitectura RL **13**
9.1 Reformulación del MDP . . . 13
9.2 Mapeo entre Co-reguladores y Componentes RL . . . 14
9.3 Hiperparámetros Nuevos . . . 14

10 Conclusiones **14**
10.1 Contribuciones de esta Versión . . . 14
10.2 Trabajo Futuro . . . 15

1 Introducción: De Grafo Dinámico a Sistema Evolutivo con Memoria

1.1 Motivación

Las versiones previas del Núcleo Lógico Evolutivo (NLE) establecieron una arquitectura en la que un grafo categórico de skills $\mathcal{G}_t = (\mathcal{S}_t, \mathrm{Mor}_t, w_t)$ evoluciona bajo el control de un agente de aprendizaje por refuerzo. Sin embargo, esta formulación presenta dos limitaciones fundamentales:

(i) **Ausencia de jerarquía formal.** Las operaciones atómicas (`add_node`, `merge`, `split`, `reweight`) modifican el grafo sin una teoría explícita de cómo objetos de distintos niveles de complejidad se relacionan, ni cómo la composición de skills simples produce skills genuinamente más complejos.

(ii) **Plasticidad y robustez sin fundamento categórico.** La noción de "emergencia" se definía mediante métricas heurísticas (clustering espontáneo, puentes conceptuales), pero carecía de una base matemática que garantice que la reorganización del grafo preserva invariantes estructurales.

La teoría de *Sistemas Evolutivos con Memoria* (MES, por *Memory Evolutive Systems*) de Ehresmann & Vanbremeersch [1] proporciona exactamente el marco categórico necesario para resolver ambas limitaciones. En este marco, un sistema complejo se modela como una *categoría jerárquica evolutiva* donde:

- Los objetos de nivel superior son **colímites** de *patrones* de objetos de niveles inferiores.
- La evolución procede mediante **complejificaciones** respecto a *opciones*.
- Una red de **co-reguladores** controla la dinámica local, cada uno operando a su propia escala temporal.
- Una **memoria** interna almacena registros de experiencias pasadas y procedimientos aprendidos.

Principio de Diseño

El grafo categórico de skills $\mathcal{G}_t$ no es meramente un grafo que evoluciona: es la *categoría de configuración* en el tiempo t de un sistema evolutivo con memoria, cuya dinámica está comandada por co-conos colímite y cuya plasticidad emerge del *principio de multiplicidad*.

1.2 Estructura del Documento

Este documento extiende la versión 6.0 del NLE incorporando sistemáticamente los conceptos de Ehresmann & Vanbremeersch. La Sección 2 presenta los preliminares categóricos del marco MES. La Sección 3 reformula el grafo de skills como sistema evolutivo jerárquico. La Sección 4 introduce la red de co-reguladores. La Sección 5 define la arquitectura de memoria. La Sección 6 formaliza la evolución del grafo como complejificación. La Sección 7 analiza plasticidad y robustez. La Sección 8 establece propiedades formales. La Sección 9 integra con la arquitectura RL existente. La Sección 10 concluye.

2 Preliminares: El Marco MES de Ehresmann & Vanbremeersch

Presentamos las definiciones fundamentales del marco MES, siguiendo [1], adaptadas a la notación del NLE.

2.1 Patrones y Colímites

Definición 2.1 (Patrón [1, Cap. 2]). Sea $\mathcal{K}$ una categoría. Un *patrón* P en $\mathcal{K}$ es un funtor

$$P : I \to \mathcal{K}$$

desde una categoría de índices finita I hacia $\mathcal{K}$. Los objetos $P_i = P(i)$ para $i \in \mathrm{Ob}(I)$ son los *componentes* del patrón, y las imágenes de los morfismos de I son los *enlaces distinguidos* del patrón.

Definición 2.2 (Colímite de un patrón [1, Cap. 2]). Un *colímite* del patrón $P : I \to \mathcal{K}$ es un objeto $cP \in \mathrm{Ob}(\mathcal{K})$ junto con un *co-cono*

$$(c_i : P_i \to cP)_{i \in \mathrm{Ob}(I)}$$

tal que:

(a) **Compatibilidad:** Para todo morfismo $d : i \to j$ en I, se cumple $c_j \circ P(d) = c_i$.

(b) **Universalidad:** Para todo objeto B y toda familia compatible $(g_i : P_i \to B)_{i \in \mathrm{Ob}(I)}$, existe un único morfismo $h : cP \to B$ tal que $h \circ c_i = g_i$ para todo i.

$$\begin{tikzcd}
P_i \arrow[r, "P(d)"] \arrow[dr, "c_i"'] \arrow[drr, bend left, "g_i"] & P_j \arrow[d, "c_j"] \arrow[dr, "g_j"] & \\
& cP \arrow[r, dashed, "\exists!\, h"'] & B
\end{tikzcd}$$

Figure 1: Co-cono colímite: la propiedad universal garantiza que toda acción colectiva del patrón se factoriza únicamente a través de cP.

Conexión con Ehresmann–Vanbremeersch

En el NLE, un patrón P corresponde a un sub-grafo de skills interconectados que, al actuar coherentemente, resuelven una clase de problemas. El colímite cP es el *skill compuesto emergente* que integra la funcionalidad colectiva del patrón.

2.2 Categorías Jerárquicas

Definición 2.3 (Categoría jerárquica [1, Cap. 3]). Una *categoría jerárquica* es una categoría $\mathcal{K}$ cuyos objetos están particionados en una secuencia finita de *niveles* $0, 1, \ldots, N$, de modo que todo objeto C de nivel $n+1$ es el colímite en $\mathcal{K}$ de al menos un patrón P cuyos componentes son de nivel $\leq n$.

Los objetos de nivel 0 son los *componentes elementales* (átomos).

Definición 2.4 (Orden de complejidad [1, Cap. 3]). El *orden de complejidad* de un objeto C en una categoría jerárquica es el menor entero n tal que C es de nivel n.

2.3 El Principio de Multiplicidad

Definición 2.5 (Patrones homólogos [1, Cap. 3]). Dos patrones P y P' en $\mathcal{K}$ son *homólogos* si sus campos operativos son isomorfos; es decir, si existe una correspondencia biunívoca entre sus enlaces colectivos que preserva composición. Intuitivamente, P y P' son funcionalmente equivalentes aunque tengan componentes distintos.

Definición 2.6 (Principio de multiplicidad [1, Cap. 3]). Una categoría jerárquica $\mathcal{K}$ satisface el *principio de multiplicidad* si existen patrones homólogos P y P' con el mismo colímite $cP = cP'$, pero que no están conectados por un cluster. Es decir, existen múltiples descomposiciones funcionalmente equivalentes para ciertos objetos complejos.

Justificación

El principio de multiplicidad es la condición necesaria y suficiente para la existencia de objetos de complejidad estrictamente creciente [1, Teorema de reducción, Cap. 3]. En el NLE, esto significa que un mismo skill compuesto puede ser realizado por distintas combinaciones de sub-skills, lo cual es exactamente la *robustez* que buscamos: si un camino de skills falla, existe un camino homólogo alternativo.

2.4 Sistemas Evolutivos

Definición 2.7 (Sistema evolutivo [1, Cap. 5]). Un *sistema evolutivo* $\mathbb{K}$ sobre una escala temporal T consiste en:

(a) Para cada $t \in T$, una categoría de configuración $\mathcal{K}_t$.

(b) Para cada par $t < t'$ en T, un *funtor de transición* (parcial):

$$k_{t,t'} : \mathcal{K}_t \rightharpoonup \mathcal{K}_{t'}$$

que modela cómo los componentes en el tiempo t se transforman (o desaparecen) en el tiempo t'.

(c) Compatibilidad: $k_{t',t''} \circ k_{t,t'} = k_{t,t''}$ cuando está definido.

Definición 2.8 (Sistema evolutivo jerárquico [1, Cap. 5]). Un sistema evolutivo $\mathbb{K}$ es *jerárquico* si cada categoría de configuración $\mathcal{K}_t$ es una categoría jerárquica, y los funtores de transición preservan los niveles (o pueden crear niveles nuevos).

2.5 Complejificación respecto a una Opción

Definición 2.9 (Opción [1, Cap. 4]). Una *opción* Op sobre una categoría $\mathcal{K}$ es una lista de objetivos de los siguientes tipos:

(i) **Absorción:** Un grafo U de elementos externos a incorporar.

(ii) **Eliminación:** Un conjunto V de objetos de $\mathcal{K}$ a eliminar.

(iii) **Ligadura (binding):** Un conjunto de patrones sin colímite en $\mathcal{K}$ que deben adquirir uno; o patrones cuyo colímite debe preservarse.

(iv) **Clasificación:** Patrones que deben adquirir un límite (clasificador).

Teorema 2.10 (Complejificación [1, Teorema 1, Cap. 4]). *Dada una categoría $\mathcal{K}$ y una opción* Op *sobre ella, existe un funtor parcial $p : \mathcal{K} \rightharpoonup \mathcal{K}'$ hacia una categoría $\mathcal{K}'$ (la* complejificación*) tal que:*

(a) Los objetivos de Op *se realizan en $\mathcal{K}'$.*

(b) Si dos patrones que Op *requiere ligar son homólogos en $\mathcal{K}$, adquieren el mismo colímite en $\mathcal{K}'$.*

(c) p satisface una propiedad universal: todo otro funtor que realice los mismos objetivos se factoriza únicamente a través de p.

Además, si $\mathcal{K}$ satisface el principio de multiplicidad, también lo hace $\mathcal{K}'$.

Teorema 2.11 (Complejificación iterada [1, Teorema 3, Cap. 4]). *Si $\mathcal{K}$ satisface el principio de multiplicidad, una secuencia de complejificaciones no puede reducirse a una única complejificación. Esto conduce a la emergencia de una jerarquía de objetos con complejidad estrictamente creciente.*

2.6 Sistemas Evolutivos con Memoria y Co-reguladores

Definición 2.12 (Sistema evolutivo con memoria [1, Cap. 6]). Un *sistema evolutivo con memoria* (MES) es un sistema evolutivo jerárquico $\mathbb{K}$ sobre una escala temporal continua T, en el que se distinguen los siguientes subsistemas evolutivos:

(a) **La memoria** MEM: Subsistema evolutivo jerárquico que se desarrolla con el tiempo, posiblemente formando niveles cada vez más altos mediante complejificaciones sucesivas.

(b) **Red de co-reguladores:** Una red de subsistemas evolutivos $\{\mathrm{CR}_k\}_{k\in K}$. Los componentes de un co-regulador se denominan sus *agentes*. Cada CR_k:

  (i) Opera a su propio nivel de complejidad.

  (ii) Posee su propia escala temporal discreta.

  (iii) Tiene acceso a componentes específicos de la memoria, llamados sus *procedimientos admisibles*.

  (iv) Envía comandos a componentes del sistema llamados *efectores*.

Definición 2.13 (Paisaje de un co-regulador [1, Cap. 6]). El *paisaje* $\mathrm{Land}(\mathrm{CR}_k, t)$ de un co-regulador CR_k en el tiempo t es una categoría que recolecta la información parcial que CR_k recibe del sistema en ese instante. No es parte del sistema mismo, sino un *modelo interno* del sistema para CR_k.

3 El Grafo de Skills como Sistema Evolutivo Jerárquico

3.1 Reformulación del Grafo Categórico

Reformulamos la categoría de skills $\mathbf{Skill}_t$ (anteriormente denotada $\mathcal{G}_t$) como una *categoría jerárquica* en el sentido de Ehresmann.

Definición 3.1 (Categoría jerárquica de skills). La categoría jerárquica de skills $\mathbf{Skill}_t$ en el tiempo t es una categoría jerárquica con niveles $0, 1, \ldots, N_t$ donde:

(a) **Nivel 0 (átomos):** Skills atómicos correspondientes a los cuatro pilares fundacionales:

$$\mathrm{Ob}(\mathbf{Skill}_t^0) = \mathcal{F}^0_{\mathrm{Set}} \cup \mathcal{F}^0_{\mathrm{Cat}} \cup \mathcal{F}^0_{\mathrm{Log}} \cup \mathcal{F}^0_{\mathrm{Type}}$$

Estos son axiomas individuales, reglas de inferencia básicas, definiciones primitivas.

(b) **Nivel 1 (clusters básicos):** Skills que son colímites de patrones de nivel 0:

$$s \in \mathrm{Ob}(\mathbf{Skill}_t^1) \iff \exists P : I \to \mathbf{Skill}_t^0 \text{ tal que } s = \operatorname{colim} P$$

Ejemplo: el skill "ZFC-axioms" es el colímite del patrón formado por los axiomas individuales de ZFC con sus relaciones de dependencia.

(c) **Nivel $n+1$:** Skills compuestos que son colímites de patrones con componentes de nivel $\leq n$:

$$s \in \mathrm{Ob}(\mathbf{Skill}_t^{n+1}) \iff \exists P : I \to \bigcup_{k\leq n} \mathbf{Skill}_t^k \text{ tal que } s = \operatorname{colim} P$$

(d) **Morfismos:** Los cuatro tipos definidos en versiones previas (dependencia $\hookrightarrow$, especialización $\twoheadrightarrow$, analogía $\leftrightarrow$, traducción $\rightsquigarrow$), enriquecidos ahora con:

- *Enlaces simples* g: factorizan a través de clusters entre patrones.
- *Enlaces complejos*: composiciones de enlaces simples que no factorizan a través de un único cluster.

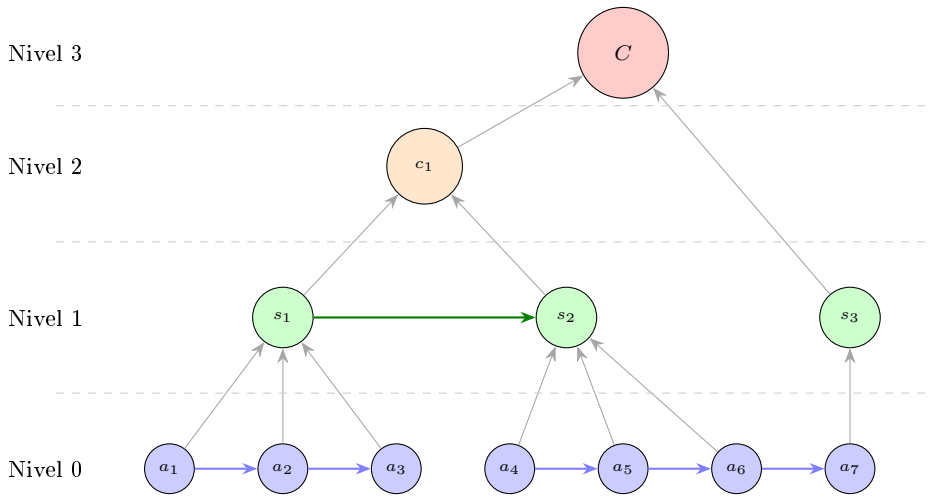


Figure 2: Categoría jerárquica de skills. Los objetos a_i de nivel 0 son skills atómicos. Las flechas grises son los morfismos del co-cono colímite. Cada nivel integra patrones del nivel inferior.

3.2 El Sistema Evolutivo de Skills

Definición 3.2 (Sistema evolutivo de skills)**.** El *sistema evolutivo de skills* $\mathbb{S}$ es un sistema evolutivo jerárquico sobre la escala temporal discreta $T = \mathbb{N}$, definido por:

(a) $\mathbf{Skill}_t$: Categoría jerárquica de configuración en el paso t.

(b) $k_{t,t+1} : \mathbf{Skill}_t \rightharpoonup \mathbf{Skill}_{t+1}$: Funtor de transición (parcial) que modela los cambios producidos por las acciones del núcleo en el paso t.

(c) $k_{t,t+1}$ es inducido por la complejificación de $\mathbf{Skill}_t$ respecto a la opción Op_t seleccionada por el agente RL (véase Sección 6).

Ejemplo 3.3 (Niveles concretos en el NLE)**.**

Nivel	Nombre	Contenido
0	Átomos	Axiomas individuales (Ext, Par, Unión, ...), reglas básicas ($\to$-intro, $\forall$-elim), constructores de tipo (Prod, Sum)
1	Clusters	ZFC-axioms, FOL=-rules, CIC-basics, Cat-basics
2	Habilidades	Inducción tranfinita, semántica de Tarski, Curry–Howard, Adjunciones
3	Competencias	Forcing, Topos internos, Verificación en Lean 4
≥ 4	Meta-skills	Traducción entre pilares, resolución de problemas abiertos

3.3 Multiplicidad en el Grafo de Skills

Proposición 3.4 (El grafo de skills satisface el principio de multiplicidad)**.** *Sea* $\mathbf{Skill}_t$ *la categoría jerárquica de skills con los cuatro pilares fundacionales. Entonces* $\mathbf{Skill}_t$ *satisface el principio de multiplicidad.*

Esbozo. Por construcción, los morfismos de traducción $\tau_1, \ldots, \tau_6$ entre los cuatro pilares (Def. 3.14 de la v6.0) aseguran que existen patrones homólogos no conectados. Por ejemplo, el concepto de "producto" admite al menos dos patrones homólogos:

- P_{Set}: producto cartesiano $A \times B$ vía la axiomática ZFC.
- P_{Cat}: producto categórico vía la propiedad universal en **Cat**.

Ambos patrones son funcionalmente equivalentes (tienen los mismos enlaces colectivos hacia otros skills), pero sus componentes pertenecen a pilares distintos y no están conectados por un cluster directo. El colímite común es el meta-skill "producto (concepto unificado)". Análogamente para "inducción/recursión", "cuantificación/tipos dependientes", etc. □

Corolario 3.5. *Por el Teorema 2.11, el sistema evolutivo de skills admite la emergencia de skills de complejidad arbitrariamente creciente mediante complejificaciones sucesivas.*

4 Red de Co-reguladores del NLE

4.1 Del Agente RL Monolítico a una Red de Co-reguladores

En la formulación previa, un único agente RL controlaba toda la dinámica del grafo. Siguiendo [1, Cap. 6], reemplazamos esta arquitectura monolítica por una *red de co-reguladores*, cada uno operando a su propio nivel de complejidad y escala temporal.

Definición 4.1 (Red de co-reguladores del NLE)**.** La red de co-reguladores $\{\text{CR}_k\}_{k=1}^4$ del NLE consiste en:

CR1: Co-regulador táctico (CR_{tac})

- *Nivel:* 0–1 (opera sobre skills atómicos y clusters básicos).
- *Escala temporal:* Rápida (cada interacción usuario).
- *Agentes:* Skills individuales de los pilares fundacionales.
- *Procedimientos:* `select`, `compose`, `translate`.
- *Efectores:* Interfaz LLM ↔ Lean 4.

CR2: Co-regulador organizativo (CR_{org})

- *Nivel:* 1–2 (opera sobre clusters y habilidades).
- *Escala temporal:* Media (cada k interacciones).
- *Agentes:* Clusters de skills y sus relaciones.
- *Procedimientos:* `merge`, `split`, `reweight`, `add_bridge`.
- *Efectores:* Motor de reorganización del grafo.

CR3: Co-regulador estratégico (CR_{str})

- *Nivel:* 2–3 (opera sobre competencias y meta-skills).
- *Escala temporal:* Lenta (cada K sesiones).
- *Agentes:* Competencias de alto nivel, traducciones inter-pilar.

- *Procedimientos:* `create_level`, `complexify`, `form_concept`.
- *Efectores:* Motor de complejificación.

CR4: Co-regulador de integridad ($\mathrm{CR}_{\mathrm{int}}$)

- *Nivel:* Transversal (todos los niveles).
- *Escala temporal:* Periódica.
- *Agentes:* Invariantes del sistema (Axiomas 7.1–7.3 de la v6.0).
- *Procedimientos:* `verify_connectivity`, `check_coverage`, `validate_consistency`.
- *Efectores:* Sistema de reparación de fracturas.

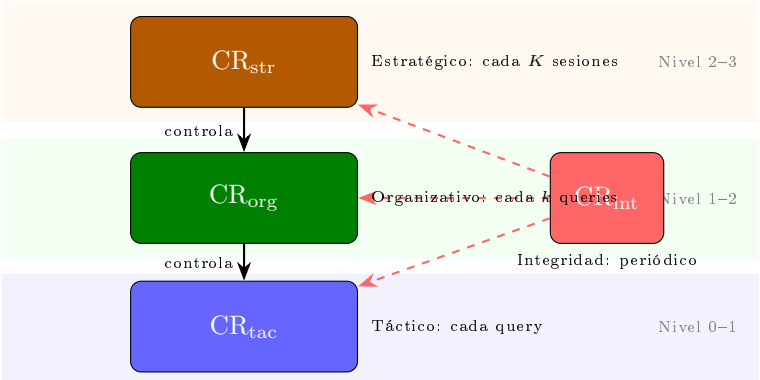


Figure 3: Red de co-reguladores. Las flechas sólidas indican control jerárquico (co-reguladores de nivel superior coordinan a los inferiores). Las flechas punteadas rojas indican las verificaciones transversales de $\mathrm{CR}_{\mathrm{int}}$.

4.2 Paisajes de los Co-reguladores

Cada co-regulador opera según su *paisaje*: una vista parcial del sistema.

Definición 4.2 (Paisajes del NLE)**.** Los paisajes de los co-reguladores del NLE son:

$$\mathrm{Land}(\mathrm{CR}_{\mathrm{tac}}, t) = \text{Subcategoría de } \mathbf{Skill}_t \text{ relevante a la query actual} \tag{1}$$
$$\mathrm{Land}(\mathrm{CR}_{\mathrm{org}}, t) = \text{Estadísticas de uso, pesos, métricas de coherencia sobre } \mathbf{Skill}_t \tag{2}$$
$$\mathrm{Land}(\mathrm{CR}_{\mathrm{str}}, t) = \text{Estructura de niveles, traducciones inter-pilar, patrones emergentes} \tag{3}$$
$$\mathrm{Land}(\mathrm{CR}_{\mathrm{int}}, t) = \text{Estado de invariantes, registros de fracturas y reparaciones} \tag{4}$$

4.3 Ciclo Operativo de un Co-regulador

Siguiendo [1, Sección 4, Cap. 6], cada co-regulador ejecuta un ciclo de cuatro fases:

Fase 1: Construcción del paisaje (decodificación). CR_k recoge información parcial del sistema para formar $\mathrm{Land}(\mathrm{CR}_k, t)$.

Fase 2: Selección de objetivos. En base al paisaje, CR_k selecciona una opción $\mathrm{Op}_k(t)$ con objetivos específicos.

Fase 3: Codificación del procedimiento. CR_k selecciona de la memoria un procedimiento admisible π_k y envía comandos a sus efectores.

Fase 4: Evaluación. Comparación del resultado con el paisaje anticipado; actualización de la memoria.

Conexión con Ehresmann–Vanbremeersch

Las cuatro fases del ciclo de un co-regulador se corresponden con las etapas del loop principal del agente RL definido en la v4.0/v6.0:

Fase MES	Correspondencia RL	Función
Construcción del paisaje	`observar_estado`(x_t)	Codificación del estado
Selección de objetivos	$a_t \sim \pi_\theta(\cdot\|x_t)$	Política del agente
Codificación	`ejecutar_acción`(a_t)	Actuación sobre el grafo
Evaluación	$r_t = R(x_t, a_t, x_{t+1})$	Función de recompensa

La diferencia clave es que en el marco MES, estos ciclos ocurren *en paralelo* a múltiples escalas temporales, mientras que el loop RL previo era secuencial y monolítico.

5 Arquitectura de Memoria del NLE

5.1 Estructura de la Memoria

Definición 5.1 (Memoria del NLE como subsistema evolutivo)**.** La *memoria* MEM_t del NLE es un subsistema evolutivo jerárquico de $\mathbb{S}$, que almacena:

(a) **Registros de experiencia** (records): Para cada interacción exitosa, un registro M que codifica la consulta, los skills utilizados, y el resultado.

(b) **Procedimientos** (procedures): Secuencias de acciones que resolvieron clases de problemas, almacenadas como patrones en MEM_t.

(c) **Conceptos-E** (E-concepts): Clases de invariancia que agrupan registros funcionalmente equivalentes (Def. 5.3).

Definición 5.2 (Registro de experiencia)**.** Un *registro* en MEM_t es un objeto M junto con:

(i) Un patrón P_M en $\mathbf{Skill}_t$ que describe los skills activados.

(ii) Un enlace $\ell_M : cP_M \to$ resultado que conecta el colímite del patrón con el outcome.

(iii) Una marca temporal $\tau_M \in T$.

(iv) Un valor de éxito $v_M \in [-1, 1]$.

Definición 5.3 (E-concepto [1, Cap. 8])**.** Dado un co-regulador CR_k y un registro M, la *clase de E-invariancia* de M respecto a CR_k es el conjunto de todos los registros M' tales que CR_k no distingue entre M y M' (producen la misma respuesta funcional).

El *E-concepto* $C_E(M)$ es el colímite en la memoria semántica del patrón formado por los representantes de la clase de invariancia, ligados por los enlaces de equivalencia funcional.

Teorema 5.4 (Preservación de colímites por el funtor E-concepto [1, Teorema 5, Cap. 8])**.** *El funtor parcial* $C_E : \mathrm{MEM} \to \text{E-Sem}$ *preserva colímites: si* $M = \operatorname{colim} Q$ *para un patrón* Q *de registros* (M_i)*, y si* M *y los* M_i *tienen E-conceptos, entonces* $C_E(M) = \operatorname{colim} C_E(Q)$.

Justificación

La preservación de colímites garantiza que los conceptos formados a partir de registros compuestos son coherentes con los conceptos de los sub-registros. En el NLE, esto significa que un meta-skill aprendido por composición de sub-skills hereda las clasificaciones semánticas de sus componentes, permitiendo la búsqueda jerárquica eficiente en la memoria.

5.2 Tipos de Memoria en el NLE

Siguiendo la clasificación de [1, Cap. 8]:

Tipo de memoria	Contenido en MES	Realización en NLE
Procedimental	Secuencias de acciones exitosas	Políticas parciales π_k de los co-reguladores, cadenas de composición de skills
Semántica	E-conceptos y sus relaciones	Clases de equivalencia de patrones de skills; índice semántico sobre el grafo
Consolidada	Registros fortalecidos con el tiempo	Skills con peso $w(f)$ alto; coconos frecuentemente activados
Empírica	Registros de experiencias concretas	Log de interacciones con timestamps, queries, resultados

6 Evolución del Grafo como Complejificación

6.1 Las Acciones del Núcleo como Opciones

La correspondencia central de esta sección es:

Principio de Diseño

Cada ciclo de acción del núcleo RL produce una *opción* Op_t sobre $\mathbf{Skill}_t$. La transición $\mathbf{Skill}_t \to \mathbf{Skill}_{t+1}$ es la *complejificación* de $\mathbf{Skill}_t$ respecto a Op_t.

Definición 6.1 (Opción inducida por acción del núcleo)**.** Dada una acción $a_t \in \mathcal{A}$ del agente RL, la opción $\mathrm{Op}(a_t)$ sobre $\mathbf{Skill}_t$ se define por:

Acción RL	Tipo de objetivo MES	Efecto
`add_node`(s_{new})	Absorción	Incorporar grafo externo $\{s_{\text{new}}\}$
`merge`(s_1, s_2)	Ligadura (binding)	Ligar el patrón $\{s_1, s_2\}$ en un colímite s_{new}
`split`(s)	Eliminación + Absorción	Eliminar s; absorber $\{s_1, s_2\}$
`add_edge`(s_1, s_2, f)	Absorción de enlace	Incorporar nuevo morfismo
`reweight`(f, w')	Preservación	Preservar estructura; modificar etiqueta
`strengthen`(f)	Preservación	Fortalecer colímite existente

6.2 La Operación `merge` como Ligadura de Patrón

La operación más importante desde el punto de vista MES es `merge`, pues corresponde a la *ligadura de un patrón*: la creación de un colímite para un patrón que no lo tenía.

Definición 6.2 (Merge como ligadura)**.** Sean $s_1, s_2 \in \mathrm{Ob}(\mathbf{Skill}_t)$ con morfismos distinguidos entre ellos. Sea $P = \{s_1 \xrightarrow{f} s_2\}$ un patrón en $\mathbf{Skill}_t$. La operación `merge`(s_1, s_2) produce:

(i) Un nuevo objeto $s_{\text{new}} = \operatorname{colim} P$ en $\mathbf{Skill}_{t+1}$.

(ii) Un co-cono $(c_1 : s_1 \to s_{\text{new}},\ c_2 : s_2 \to s_{\text{new}})$.

(iii) Para cada morfismo $g : s_i \to B$ en $\mathbf{Skill}_t$, un único $h : s_{\text{new}} \to B$ tal que $h \circ c_i = g$ (propiedad universal).

Esto es exactamente la construcción de la complejificación simple ([1, Sección 3.2, Cap. 4]).

$$\begin{tikzcd}
s_1 \arrow[r, bend left, "f"] \arrow[dr, "c_1"'] & s_2 \arrow[d, "c_2"] \arrow[dr, bend left, "g_2"] \arrow[r, bend left, "g_1"] & {} \\
& s_{\text{new}} = \operatorname{colim} P \arrow[r, dashed, "\exists!\, h"'] & B
\end{tikzcd}$$

Figure 4: La operación `merge` como ligadura de patrón. El skill fusionado s_{new} es el colímite del patrón (s_1, s_2, f), y hereda enlaces universales hacia todos los objetos B del grafo.

6.3 Emergencia de Enlaces Complejos

Definición 6.3 (Enlace simple y enlace complejo en $\mathbf{Skill}_t$)**.** En la complejificación $\mathbf{Skill}_{t+1}$:

- Un *enlace simple* es un morfismo que factoriza a través de un único cluster entre patrones de $\mathbf{Skill}_t$.
- Un *enlace complejo* es un morfismo que es composición de enlaces simples que no factorizan a través de clusters adyacentes.

Conexión con Ehresmann–Vanbremeersch

Los enlaces complejos son exactamente las *propiedades emergentes* del sistema: relaciones entre skills compuestos que no son deducibles de las relaciones entre sus componentes individuales, sino que surgen de la estructura global de la complejificación. Esto formaliza categóricamente la noción de emergencia que en la v6.0 se definía sólo mediante métricas heurísticas.

7 Plasticidad y Robustez vía el Principio de Multiplicidad

7.1 Robustez por Descomposiciones Múltiples

Teorema 7.1 (Robustez del grafo de skills)**.** *Sea $s \in \operatorname{Ob}(\mathbf{Skill}_t)$ un skill de nivel $n \geq 1$ en una categoría que satisface el principio de multiplicidad. Si una descomposición P de s sufre la eliminación de un componente P_i, entonces existe al menos un patrón homólogo P' tal que $s = \operatorname{colim} P'$ y $P_i \notin \operatorname{Ob}(P')$.*

Esbozo. Por el principio de multiplicidad, s admite patrones homólogos no conectados. Si P_i es eliminado, el colímite s sobrevive si existe P' homólogo a P que no contiene P_i. La condición de que los patrones no estén conectados por un cluster garantiza que P' es independiente de P_i. □

Justificación

Esto resuelve un problema práctico concreto del NLE: si un skill deja de estar disponible (por ejemplo, una dependencia de Mathlib se depreca), el sistema puede redirigir las consultas a través de un patrón homólogo alternativo, sin pérdida de funcionalidad. La robustez no es accidental sino *estructural*, garantizada por el principio de multiplicidad.

7.2 Plasticidad por Complejificación

Definición 7.2 (Plasticidad categórica)**.** La *plasticidad* del sistema $\mathbb{S}$ en el tiempo t es la capacidad de formar nuevos colímites para patrones que previamente carecían de uno. Formalmente, es la cardinalidad del conjunto:

$$\operatorname{Plast}(t) = \{P \text{ patrón en } \mathbf{Skill}_t \mid P \text{ no tiene colímite en } \mathbf{Skill}_t, \text{ pero lo adquiere en } \mathbf{Skill}_{t+1}\}$$

Proposición 7.3 (La complejificación preserva plasticidad)**.** *Si* $\mathbf{Skill}_t$ *satisface el principio de multiplicidad y* Op_t *incluye la preservación de colímites existentes, entonces* $\mathbf{Skill}_{t+1}$ *también satisface el principio de multiplicidad (por el Teorema 2.10), y por tanto mantiene su potencial de plasticidad futura.*

7.3 Fracturas y Reparación

Definición 7.4 (Fractura [1, Cap. 7])**.** Una *fractura* en el sistema $\mathbb{S}$ en el tiempo t ocurre cuando un co-regulador CR_k selecciona un procedimiento π_k cuyo resultado difiere del paisaje anticipado. Formalmente, si $\mathrm{Land}^{\mathrm{ant}}(\mathrm{CR}_k, t+1)$ es el paisaje anticipado y $\mathrm{Land}(\mathrm{CR}_k, t+1)$ es el paisaje real:

$$\mathrm{Fractura}(\mathrm{CR}_k, t) \iff \mathrm{Land}^{\mathrm{ant}}(\mathrm{CR}_k, t+1) \ncong \mathrm{Land}(\mathrm{CR}_k, t+1)$$

Definición 7.5 (Discronía)**.** Una *discronía* ocurre cuando dos co-reguladores operan a escalas temporales incompatibles, causando conflictos en sus procedimientos. En el NLE:

- $\mathrm{CR}_{\mathrm{tac}}$ modifica skills rápidamente $\to$ $\mathrm{CR}_{\mathrm{org}}$ no detecta los cambios a tiempo.
- $\mathrm{CR}_{\mathrm{str}}$ inicia una complejificación $\to$ $\mathrm{CR}_{\mathrm{tac}}$ accede a skills en estado transitorio.

La reparación de fracturas es responsabilidad del co-regulador de integridad $\mathrm{CR}_{\mathrm{int}}$, que:

(i) Detecta la fractura comparando invariantes antes y después de cada transición.

(ii) Clasifica la fractura (temporal, estructural, lógica).

(iii) Selecciona un procedimiento de reparación de la memoria.

(iv) Ejecuta la reparación y verifica el resultado.

8 Propiedades Formales

8.1 Axiomas Extendidos

Los axiomas de la v6.0 se enriquecen:

Axioma 8.1 (Jerarquía). Para todo $t \geq 0$, $\mathbf{Skill}_t$ es una categoría jerárquica con al menos 2 niveles.

Axioma 8.2 (Multiplicidad). Para todo $t \geq 0$, $\mathbf{Skill}_t$ satisface el principio de multiplicidad (Def. 2.6).

Axioma 8.3 (Conectividad (v6.0, Axioma 7.1, reforzado)). Para todo $t \geq 0$, $\mathbf{Skill}_t$ es débilmente conexa *y* cada pilar fundacional $\mathcal{F}_i$ está conectado a al menos otro pilar por un morfismo de traducción τ_j.

Axioma 8.4 (Cobertura fundacional (v6.0, Axioma 7.2, reforzado)). Para todo $s \in \mathrm{Ob}(\mathbf{Skill}_t)$, existe un patrón P con $s = \operatorname{colim} P$ (o s es atómico) y los componentes atómicos de P pertenecen a al menos un pilar $\mathcal{F}_i$.

8.2 Teoremas

Teorema 8.5 (Consistencia evolutiva)**.** *Si* $\mathbf{Skill}_t$ *es una categoría jerárquica válida que satisface el principio de multiplicidad, y* Op_t *es una opción válida (generada por una acción* $a_t \in \mathcal{A}$*), entonces* $\mathbf{Skill}_{t+1} = \mathrm{Compl}(\mathbf{Skill}_t, \mathrm{Op}_t)$ *también es una categoría jerárquica válida que satisface el principio de multiplicidad.*

Proof. Consecuencia directa del Teorema de complejificación (Teorema 2.10), que garantiza:

(i) Los objetivos se realizan en $\mathbf{Skill}_{t+1}$ (la categoría es válida).

(ii) Patrones homólogos adquieren el mismo colímite (la estructura jerárquica se preserva).

(iii) Si $\mathbf{Skill}_t$ satisface multiplicidad, $\mathbf{Skill}_{t+1}$ también.

La verificación de que cada acción a_t genera una opción válida se sigue de la Def. 6.1. □

Teorema 8.6 (Emergencia garantizada)**.** *Si* $\mathbf{Skill}_0$ *satisface el principio de multiplicidad y la política* π_θ *ejecuta al menos una operación* `merge` *de patrones con enlaces complejos cada* K *pasos, entonces la secuencia* $(\mathbf{Skill}_t)_{t\geq 0}$ *genera skills de órdenes de complejidad estrictamente crecientes.*

Proof. Por el Teorema 2.11, si $\mathbf{Skill}_t$ satisface multiplicidad y la complejificación introduce patrones cuyos enlaces distinguidos incluyen al menos un enlace complejo, entonces el colímite resultante tiene orden de complejidad estrictamente mayor que los componentes. La hipótesis de `merge` periódico garantiza que tales complejificaciones ocurren regularmente. □

Teorema 8.7 (Preservación de cobertura bajo complejificación)**.** *Si* $\mathbf{Skill}_t$ *satisface el Axioma de cobertura fundacional y* Op_t *no elimina ningún pilar fundacional ni morfismo de traducción, entonces* $\mathbf{Skill}_{t+1}$ *también satisface dicho axioma.*

Proof. Los nuevos objetos de $\mathbf{Skill}_{t+1}$ son colímites de patrones en $\mathbf{Skill}_t$. Por inducción en los niveles, los componentes de estos patrones están cubiertos por los pilares. El colímite hereda la cobertura a través del co-cono. □

9 Integración con la Arquitectura RL

9.1 Reformulación del MDP

La formulación MDP de la v6.0 se extiende incorporando la estructura MES:

Definición 9.1 (MDP extendido del NLE-MES)**.** $\mathcal{M} = (\mathcal{X}', \mathcal{A}', P, R', \gamma)$ donde:

Estado extendido:

$$x'_t = \big(\underbrace{c_L, g_{\text{Lean}}, h, m}_{\text{v6.0}},\ \underbrace{\mathbf{Skill}_t,\ \text{nivel}(s)\ \forall s,\ \text{MEM}_t,\ \text{Land}(\text{CR}_k, t)\ \forall k}_{\text{componentes MES}}\big)$$

Acciones extendidas:

$$\mathcal{A}' = \underbrace{\mathcal{A}_1 \sqcup \mathcal{A}_2 \sqcup \mathcal{A}_3}_{\text{v6.0}} \sqcup \underbrace{\mathcal{A}_4}_{\text{acciones MES}}$$

donde $\mathcal{A}_4 = \{\texttt{complexify}, \texttt{form_concept}, \texttt{create_level}, \texttt{repair_fracture}\}$.

Recompensa extendida:

$$R'(x, a) = r_{\text{task}} + \lambda_1 r_e + \lambda_2 r_{\text{org}} + \lambda_3 r_{\text{emerge}} + \underbrace{\lambda_4 \cdot r_{\text{hier}} + \lambda_5 \cdot r_{\text{mem}}}_{\text{nuevos términos MES}} \tag{5}$$

con:

$$r_{\text{hier}} = \delta_1 \cdot \Delta\text{niveles}(\mathbf{Skill}_t) + \delta_2 \cdot \mathbb{1}[\text{nuevo colímite formado}] \tag{6}$$

$$r_{\text{mem}} = \delta_3 \cdot \Delta|\text{MEM}_t| + \delta_4 \cdot \mathbb{1}[\text{E-concepto formado}] \tag{7}$$

9.2 Mapeo entre Co-reguladores y Componentes RL

Componente MES	Componente RL	Realización
Paisaje Land(CR_k, t)	Observación $x_t^{(k)}$	Encoder específico por co-regulador
Selección de objetivos	Política $\pi_\theta^{(k)}$	Cabeza actor específica
Procedimiento admisible	Acción $a_t^{(k)}$	Subred por tipo de co-regulador
Evaluación	Recompensa $r_t^{(k)}$	Término específico en R'
Memoria	Replay buffer + grafo de skills	Almacenamiento persistente

9.3 Hiperparámetros Nuevos

Símbolo	Descripción	Valor	Rango	Justificación
λ_4	Peso jerarquía	0.1	$[0.05, 0.3]$	Fomentar formación de niveles sin forzarla
λ_5	Peso memoria	0.08	$[0.03, 0.2]$	Incentivar consolidación gradual
k	Frecuencia $\mathrm{CR}_{\mathrm{org}}$	10	$[5, 50]$	Balance reactividad/estabilidad
K	Frecuencia $\mathrm{CR}_{\mathrm{str}}$	100	$[50, 500]$	Cambios estratégicos infrecuentes

10 Conclusiones

10.1 Contribuciones de esta Versión

Este documento ha establecido:

(I) **Reformulación del grafo de skills como categoría jerárquica,** con niveles formales de complejidad y colímites como mecanismo de integración.

(II) **Demostración de que el grafo satisface el principio de multiplicidad,** lo cual garantiza tanto robustez (descomposiciones alternativas) como potencial de emergencia ilimitada.

(III) **Red de co-reguladores** que reemplaza el agente RL monolítico, permitiendo regulación multi-escala: táctica, organizativa, estratégica y de integridad.

(IV) **Arquitectura de memoria** con registros, procedimientos y E-conceptos, siguiendo fielmente el marco MES.

(V) **Formalización de la evolución del grafo como complejificación categórica,** estableciendo que cada acción del núcleo induce una opción y la transición es la complejificación correspondiente.

(VI) **Teoría de plasticidad y robustez** fundamentada en el principio de multiplicidad y el teorema de complejificación iterada.

(VII) **Mecanismo de fracturas y reparación** mediante el co-regulador de integridad.

(VIII) **Teoremas de consistencia, emergencia y preservación** que extienden las propiedades formales de la v6.0 con garantías categóricas.

10.2 Trabajo Futuro

- **Implementación:** Prototipo de la red de co-reguladores como sistema multi-agente jerárquico.
- **Fibración:** Formalizar el sistema evolutivo $\mathbb{S}$ como fibración sobre la categoría de tiempos, siguiendo [1, Sección 6, Cap. 5].
- **Colímites locales:** Explorar la extensión a colímites locales [1, Apéndice] para modelar skills que admiten realizaciones contextuales diferentes.
- **Núcleo arquetipal:** Definir el análogo del *archetypal core* [1, Cap. 10] como sub-red de E-conceptos fuertemente conectados que sirva de referente estable para los co-reguladores intencionales.
- **Formalización en Lean 4:** Verificar mecánicamente los teoremas centrales (Teoremas 8.5–8.7).

Principio de Diseño

El Núcleo Lógico Evolutivo, reformulado como Sistema Evolutivo con Memoria, no sólo *usa* conocimiento matemático: lo *cultiva* mediante complejificaciones sucesivas, lo *protege* mediante el principio de multiplicidad, y lo *comprende* mediante la formación progresiva de E-conceptos. El sistema es un organismo categórico vivo.

References

[1] Ehresmann, A. C. & Vanbremeersch, J.-P. (2007). *Memory Evolutive Systems: Hierarchy, Emergence, Cognition*. Studies in Multidisciplinarity, Vol. 4. Elsevier.

[2] Ehresmann, A. C. & Vanbremeersch, J.-P. (1987). Hierarchical evolutive systems: a mathematical model for complex systems. *Bull. Math. Bio.*, 49(1), 13–50.

[3] Mac Lane, S. (1998). *Categories for the Working Mathematician*. 2nd ed., Springer.

[4] Goldblatt, R. (2006). *Topoi: The Categorial Analysis of Logic*. Dover.

[5] Schulman, J. et al. (2017). Proximal Policy Optimization Algorithms. *arXiv:1707.06347*.

[6] Fischbach, M. A. & Walsh, C. T. (2024). Problem choice and decision trees in science and engineering. *Cell*, 187, 1828–1833.

[7] Gödel, K. (1930). Die Vollständigkeit der Axiome des logischen Funktionenkalküls. *Monatshefte für Mathematik und Physik*, 37, 349–360.

[8] de Moura, L. & Ullrich, S. (2021). The Lean 4 Theorem Prover and Programming Language. *CADE-28*.

[9] Kunen, K. (2011). *Set Theory: An Introduction to Independence Proofs*. Elsevier.

[10] Martin-Löf, P. (1984). *Intuitionistic Type Theory*. Bibliopolis.

[11] Miller, D. (2025). *Proof Theory and Logic Programming: Computation as Proof Search*. Cambridge University Press.

[12] Barendregt, H. (1992). Lambda calculi with types. *Handbook of Logic in Computer Science*, Vol. 2.

[13] Edelman, G. M. (1989). *The Remembered Present: A Biological Theory of Consciousness.* Basic Books.

Núcleo Lógico Evolutivo v7.0

LLM ↔ Lean 4 ↔ Grafo Jerárquico Categórico

MES · Co-reguladores · Complejificación · Principio de Multiplicidad

Febrero 2026